General - RPS Data

NIIT TRAINING DIVISION | LTM - iRUN Full Stack Engineering

TRAINING LAB GUIDE

LAB EXERCISE - 1

Kafka Architecture — Start a KRaft Cluster and Create Topics

Start a 3-Broker Cluster · Inspect the KRaft Controller Quorum · Create Topics · Find Partitions on Disk

Module Series	Apache Kafka Operations & Streaming Platforms — Day 4
Lab Type	Guided Hands-On SOP — Kafka Platform Operations
Duration	48 minutes (5 min setup + 25 min tasks + 10 min validation + 8 min debrief)
Difficulty	⭐⭐ Beginner – Fully Guided with Conceptual Depth
Platform	Linux lab VM with Docker Engine + Compose v2; Apache Kafka 4.3.1 (apache/kafka image), 3-node KRaft cluster
Document Version	v1.0 | IT Training Division | 2026
Learner Name	_____
Date & Batch	_____
Prerequisite Lab Completed	☐ Day 1 Lab 3 (Streaming Platforms — Batch vs Real-Time) — signed off
Instructor Sign-Off	_____

1. LAB OVERVIEW

1.1 Why This Lab Exists

On Day 1 you simulated a topic with a file and offsets with a JSON file. Apache Kafka is the production system behind that idea: a cluster of brokers that store topics as replicated, partitioned, append-only logs, coordinated by a controller quorum (KRaft — Kafka 4.x no longer uses ZooKeeper). Every Kafka operations task — capacity, failover, lag, skew — starts with knowing which component holds what. In this lab you start a real three-broker cluster, inspect its controller quorum, create topics with different partition and replication settings, and find the partition logs on disk.

NIIT iRUN Alignment

INCIDENT OWNERSHIP: A producer team reports "Kafka is down". You own the first triage step: which brokers are up, which node leads the controller quorum, and are the topic's partitions online?

TROUBLESHOOTING EXCELLENCE: You will read kafka-metadata-quorum and kafka-topics output to locate every partition's leader and replicas, instead of treating the cluster as a black box.

OPERATIONAL READINESS: Topics are created explicitly with chosen partitions, replication factor and min.insync.replicas — never by accident with defaults.

1.2 Lab Scenario

Incident Briefing — Stand Up RetailCo's Event Backbone

TIME: 09:30 | PRIORITY: MEDIUM | ASSIGNED TO: You

REQUEST RECEIVED: "RetailCo is replacing nightly file transfers with Kafka. Build a three-broker development cluster that matches production's architecture, and create the first topics for orders, payments and audit events."

CONTEXT: Your lab VM has Docker. The starter files in /labfiles/day4 contain a docker-compose.yml for three Kafka 4.3.1 nodes, each acting as broker AND controller (KRaft combined mode), and a helper script ./k.

YOUR MISSION: Start the cluster, prove all three nodes are healthy, create three topics with deliberate settings, and locate their partitions on disk. You have 25 minutes for the core tasks.

Figure: Lab 1 — Starting Environment: Docker Installed, Starter Files Present, No Kafka Containers Running

1.3 Conceptual Foundation

Kafka has a small number of components. Learn each one's job — every Day 4 lab exercises all of them.

Figure: Kafka Architecture — Producers → Brokers (Topics → Partitions → Replicas) → Consumers (Groups), Coordinated by the KRaft Controller Quorum

General - RPS Data

Component	Role	In This Lab	How You Inspect It
Broker	Stores partition replicas; serves producers and consumers	kafka-1, kafka-2, kafka-3	docker compose ps; kafka-topics --describe
Controller (KRaft)	Manages cluster metadata: topics, leaders, ISR; one active leader of the quorum	Same 3 nodes (combined mode)	kafka-metadata-quorum describe --status
Topic / Partition	Named stream split into ordered, append-only logs	orders, payments, audit-log	kafka-topics --describe
Replica / Leader / ISR	Copies of a partition; one leader serves traffic; ISR = replicas in sync	Replication factor 3 or 1	Leader:, Replicas:, Isr: columns

Figure: Where Data Lives: Each Partition Replica Is a Folder of Segment Files (.log, .index, .timeindex) on a Broker

1.4 Learning Objectives

- Describe Kafka's components: brokers, KRaft controllers, topics, partitions, replicas, producers and consumers
- Start a three-node KRaft cluster with Docker Compose and confirm every node is healthy
- Inspect the controller quorum: cluster ID, active controller and voters
- Create topics with explicit partition count, replication factor and min.insync.replicas
- Read topic descriptions: partition leaders, replica lists and in-sync replicas (ISR)
- Locate partition logs and segment files on a broker's disk

1.5 What You Will NOT Do

Hard Stops — Absolute Prohibitions for This Lab

✗ NEVER edit docker-compose.yml in /labfiles — work in your own copy in ~/day4

✗ NEVER delete files inside a broker's log directory — Kafka owns them

✗ NEVER run docker compose down -v during the day — it deletes every topic (only at end-of-day clean-up)

✗ NEVER create production-style topics with replication factor 1 — audit-log is the deliberate exception for comparison

✗ NEVER proceed past Task 2 until the quorum shows three voters and a leader

General - RPS Data

2. PREREQUISITES & SETUP

2.1 Knowledge Prerequisites

Prerequisite	Where Covered	Verified?
Topics, partitions, offsets, consumer groups (concepts)	Day 1 Lab 3	☐
Linux terminal and Docker basics	Foundation Module 1	☐
Kafka architecture and components	Day 4 Theory — Session 1	☐
Brokers & replication	Day 4 Theory — Session 1	☐
KRaft (no ZooKeeper)	Day 4 Theory — Session 1	☐

2.2 Tools & Access Required

General - RPS Data

⌨ Pre-Lab — Verify tool versions and lab files

```
# Update package list and install system dependencies
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg lsb-release

# Add Docker's official GPG key
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

# Set up the Docker repository
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list >
/dev/null

# Install Docker Engine, CLI, and Docker Compose Plugin
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin

# Configure user permissions (run docker without sudo)
sudo usermod -aG docker $USER
newgrp docker

# Pre-pull Apache Kafka 4.3.1 Docker image
docker pull apache/kafka:4.3.1

# Create lab workspace directory
mkdir -p ~/day4 && cd ~/day4
```

```
# Verify Docker and Compose v2
```

```
docker --version
```

```
docker compose version
```

```

```

```
# Verify the pre-pulled image
```

```
docker image ls apache/kafka
```

2.3 Pre-Lab Environment State Verification

Run all checks below and confirm the output matches expectations before starting the timed tasks:

General - RPS Data

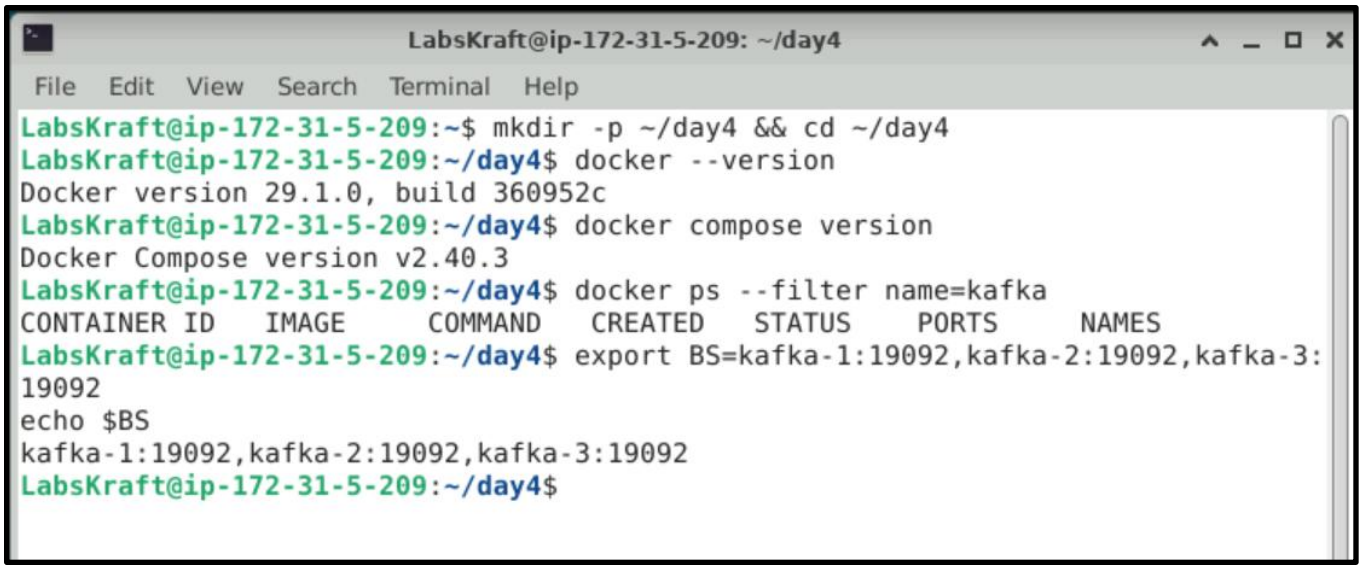
⌨ Pre-Lab — Copy files and set the bootstrap variable

```
# 1. Confirm no Kafka containers are running yet
docker ps --filter name=kafka
# Expected: only the header line

# 2. Set the bootstrap list used by every command (container-internal addresses)
export BS=kafka-1:19092,kafka-2:19092,kafka-3:19092
echo $BS
# Remember: run 'export BS=...' again in every NEW terminal you open today
```

PRE-LAB: Record your Docker and Compose versions and the apache/kafka image tag on your VM.
Answer:
_____
_____

General - RPS Data



3. CONCEPT DEEP-DIVE BEFORE STARTING

3.1 KRaft — The Controller Quorum

Kafka 4.x stores cluster metadata in an internal Raft log (__cluster_metadata) replicated across the controller nodes. One controller is the active leader; the others are voters ready to take over. A majority of voters must be available for metadata changes (creating topics, electing partition leaders). With three voters, the cluster tolerates the loss of one.

Why This Matters Operationally

Brokers keep serving existing partitions as long as each partition has a leader, but changes such as leader elections and topic creation need the controller quorum. Losing two of three controllers freezes metadata — that is why production clusters use three or five controllers.

To confirm quorum health: kafka-metadata-quorum describe --status shows LeaderId, CurrentVoters and follower lag.

3.2 The Critical Distinction: Container Running vs Broker Ready

⚠ Most Common Mistake When Starting Kafka

docker compose up returns as soon as the containers are created. Engineers immediately run commands and get timeouts or "node not available" errors, then assume the configuration is wrong.

REASON: Each node must start, join the controller quorum and register as a broker — this takes several seconds after the container starts.

YOU MUST confirm readiness from the broker logs ("Kafka Server started") and the quorum status before creating topics.

General - RPS Data

3.3 Topic Settings — Where Verification Differs

Setting	Decides	Can It Change Later?	Check Command
Partitions / Replication factor	Parallelism and key distribution / number of copies (fault tolerance)	Partitions: increase only / RF: only by reassignment	kafka-topics --describe (PartitionCount, ReplicationFactor)
min.insync.replicas	How many in-sync copies an acks=all write needs	Yes (topic config)	kafka-topics --describe (Configs:)

How to Choose Partitions and Replication Factor

Ask two questions: "How many consumers must read this topic in parallel?" → at least that many partitions. "How many broker failures must it survive without losing data?" → replication factor 3 with min.insync.replicas = 2 is the common production default (survives one failure while still accepting acks=all writes).

4. LAB TASKS

Figure: Lab 1 — Four-Task Workflow: Start Cluster → Inspect Quorum → Create Topics → Find Partitions on Disk
Figure: Expected Result — 3 Healthy Nodes, Quorum With 3 Voters, Topics orders/payments/audit-log, Segment Files Located

TASK 1 — START: Launch the Three-Node KRaft Cluster

1 **Start the Cluster and Confirm Every Node Is Ready** ⏱ 5–6 min
Kafka Ops SOP — Day 4 Lab 1

Docker Compose starts three containers. Each runs Kafka in combined mode (broker + controller). You will wait for all three to report that the Kafka server has started.

Task 1 — Start and verify

```
# Step 1: start the cluster in the background
docker compose up -d

# Step 2: container status
docker compose ps

# Step 3: readiness - repeat after 10-20 seconds until all three nodes print the line
docker compose logs | grep "Kafka Server started"

# Step 4: the helper runs Kafka tools inside kafka-1 - list topics (none yet)
./k kafka-topics.sh --bootstrap-server $BS --list
```

Expected Result

```
Step 2:  kafka-1, kafka-2, kafka-3   STATUS Up   PORTS 29092 / 39092 / 49092 -> 9092
Step 3:  kafka-1 | ... [KafkaRaftServer nodeId=1] Kafka Server started   (and the
same for nodeId=2 and nodeId=3)
Step 4:  empty output (no user topics yet)
```

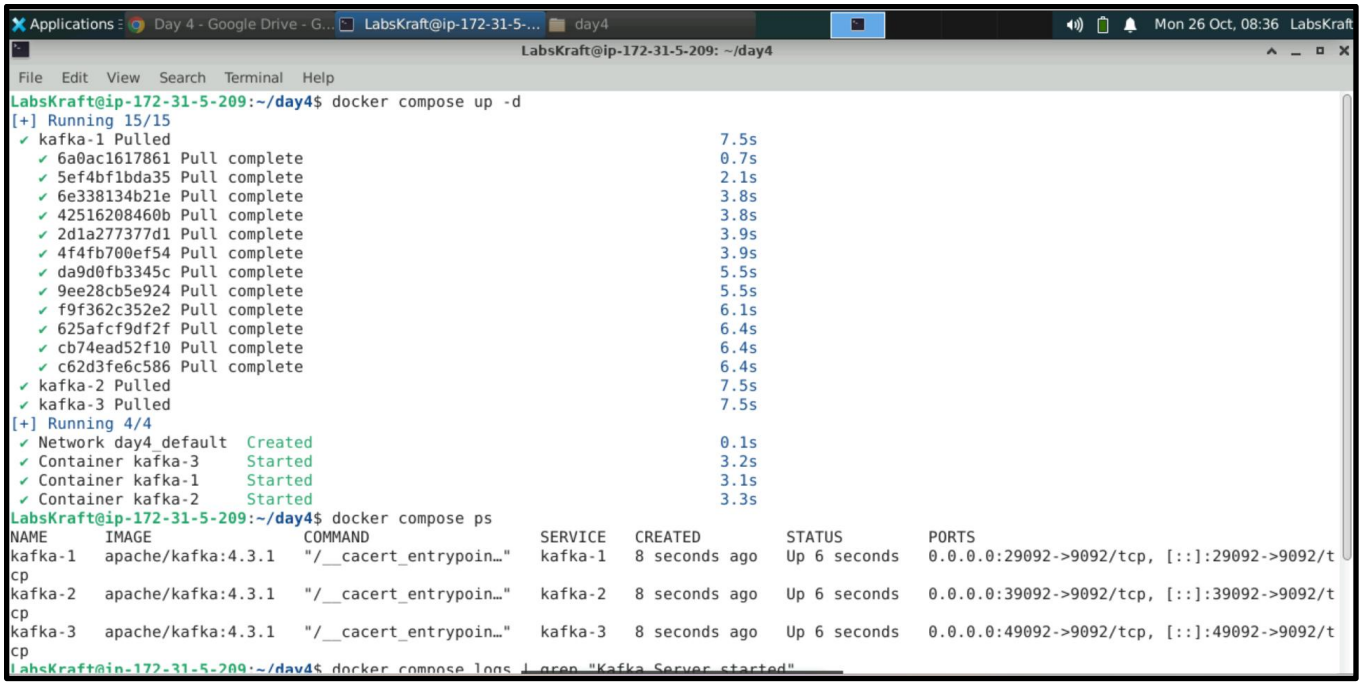
1.A Record the status and host port of each container, and how long it took until all three logged Kafka Server started. Answer:
Status: All three containers (kafka-1, kafka-2, kafka-3) are **Up**.
Host Ports:

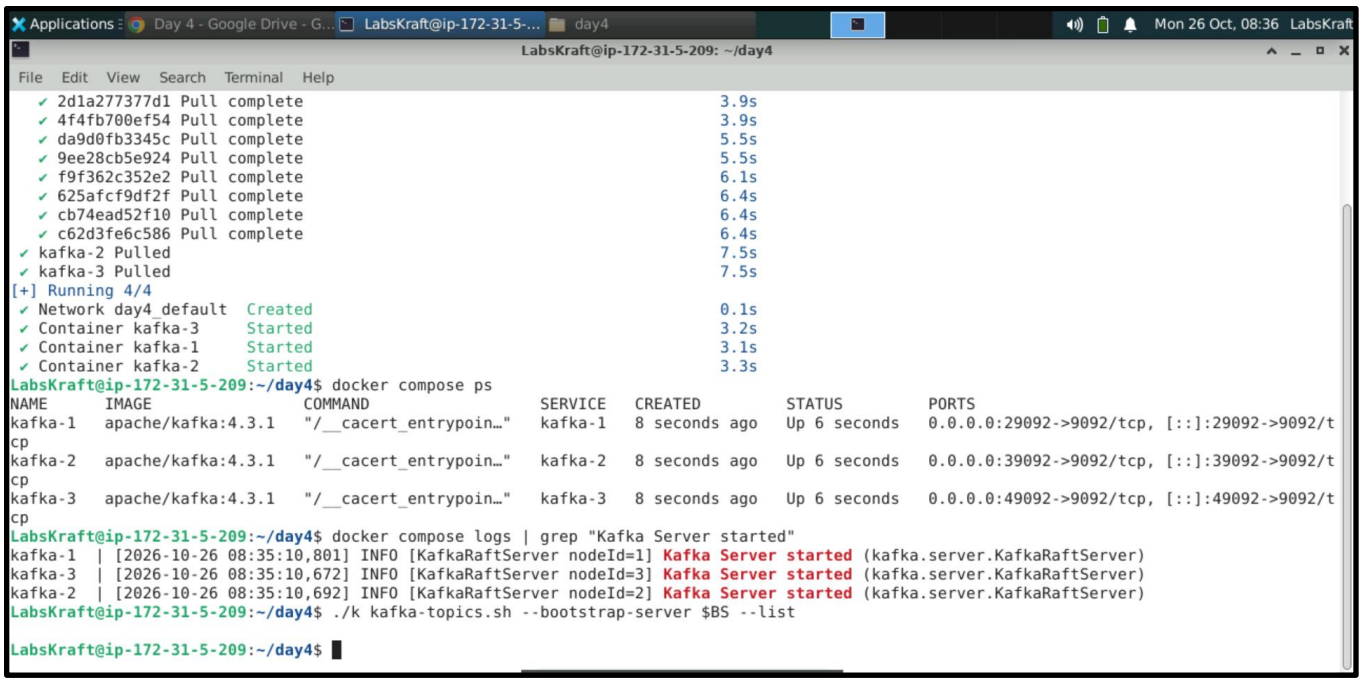
- kafka-1: 0.0.0.0:29092->9092/tcp
- kafka-2: 0.0.0.0:39092->9092/tcp

- kafka-3: 0.0.0.0:49092->9092/tcp

Startup Time: Approximately **8 seconds** after running docker compose up -d for all three nodes to successfully log Kafka Server started.

1.B ./k runs tools INSIDE kafka-1, using addresses like kafka-2:19092. Why would those addresses not work from your VM's own shell? Answer:
Addresses like `kafka-2:19092` rely on Docker's internal user-defined bridge network (`day4_default`), where container names are resolvable as hostnames exclusively *within* that network. Your VM's host shell sits outside this Docker network and does not have visibility into Docker's internal DNS resolution service, meaning the host shell cannot resolve the hostname `kafka-2`.





General - RPS Data

TASK 2 — QUORUM: Inspect the KRaft Controller Quorum

2 Read the Cluster ID, Active Controller and Voters ⏱ 5 min

Kafka Ops SOP — Day 4 Lab 1

The metadata quorum tool shows which node is the active controller and how far behind the others are.

Task 2 — Quorum status and replication

```
./k kafka-metadata-quorum.sh --bootstrap-server $BS describe --status
```

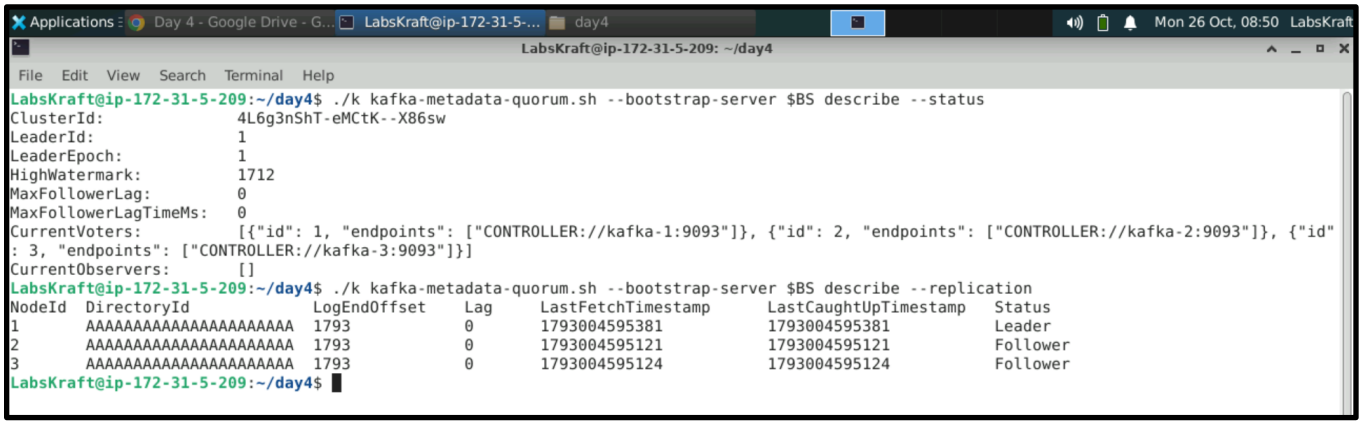
```
./k kafka-metadata-quorum.sh --bootstrap-server $BS describe --replication
```

Expected Result

```
--status:       ClusterId: 4L6g3nShT-eMCtK--X86sw | LeaderId: 1, 2 or 3 | LeaderEpoch | HighWatermark
                MaxFollowerLag: 0 (or very small) | CurrentVoters: ids 1, 2, 3 | CurrentObservers: none
--replication:  one row per node: NodeId, LogEndOffset, Lag, LastFetchTimestamp, Status (Leader / Follower)
```

2.A Record the ClusterId, LeaderId and the list of CurrentVoters. Answer:
ClusterId: 4L6g3nShT-eMCtK--X86sw
LeaderId: 1
CurrentVoters: ids 1, 2, 3

2.B How many controller nodes could fail before metadata changes stop? Explain using the word majority. Answer:
One controller node can fail. In a 3-node KRaft quorum, a **majority** (at least 2 out of 3 active voters) is strictly required to achieve consensus and commit metadata changes. If two controller nodes fail, the cluster loses its majority quorum, meaning metadata operations (such as creating topics or electing partition leaders) will freeze.



⚠ STOP CHECK — Confirm Quorum Health, Not Assumption

Before moving to Task 3, confirm CurrentVoters lists three nodes and a LeaderId is present — do not create topics on a cluster whose quorum you have not checked.

If in doubt — show your instructor the --status output before proceeding to Task 3.

TASK 3 — TOPICS: Create Topics With Deliberate Settings

3	**Create orders, payments and audit-log and Read Their Layout** *Kafka Ops SOP — Day 4 Lab 1*	⏱ 8 min

Three topics, three different designs: orders needs parallelism, payments needs durability guarantees, and audit-log is a deliberately fragile single-copy topic for comparison in Lab 3.

⌨ Task 3 — Create, try an impossible topic, list and describe

```
# Step 1: create the three topics
./k kafka-topics.sh --bootstrap-server $BS --create --topic orders    --partitions 6
--replication-factor 3
./k kafka-topics.sh --bootstrap-server $BS --create --topic payments  --partitions 3
--replication-factor 3 \
     --config min.insync.replicas=2
./k kafka-topics.sh --bootstrap-server $BS --create --topic audit-log --partitions 1
--replication-factor 1

# Step 2: try a replication factor larger than the number of brokers (must FAIL)
./k kafka-topics.sh --bootstrap-server $BS --create --topic too-many-copies --
partitions 1 --replication-factor 4

# Step 3: list and describe
./k kafka-topics.sh --bootstrap-server $BS --list
./k kafka-topics.sh --bootstrap-server $BS --describe --topic orders
./k kafka-topics.sh --bootstrap-server $BS --describe --topic payments
```

⌨ Expected Output

```
Step 1:  Created topic orders. / Created topic payments. / Created topic audit-log.
```

```
Step 2:  Error ... InvalidReplicationFactorException: replication factor 4 cannot be
reached - only 3 brokers registered
```

```
Step 3:  audit-log, orders, payments
```

```
orders:   PartitionCount: 6  ReplicationFactor: 3  then 6 rows: Partition: 0..5
Leader: n  Replicas: a,b,c  Isr: a,b,c
```

```
payments: PartitionCount: 3  ReplicationFactor: 3  Configs: min.insync.replicas=2
```

```
Leaders are spread across brokers 1, 2 and 3; every Isr lists all three replicas
```

3.A For orders, record the leader of each partition 0–5 and how many partitions each broker leads.
Answer:

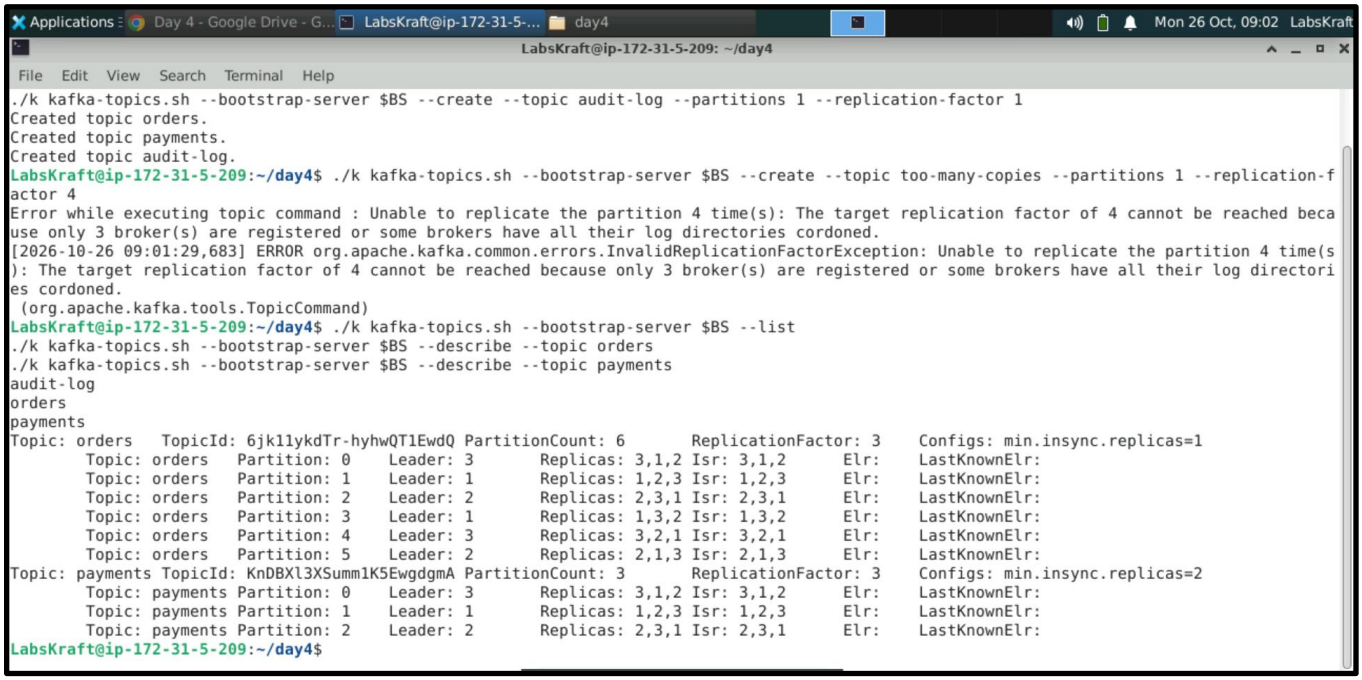
Partition Leaders:

- Partition 0: **Leader 3**
- Partition 1: **Leader 1**
- Partition 2: **Leader 2**
- Partition 3: **Leader 1**
- Partition 4: **Leader 3**
- Partition 5: **Leader 2**

Partitions Led Per Broker:

- **Broker 1:** Leads 2 partitions (Partitions 1 and 3)
- **Broker 2:** Leads 2 partitions (Partitions 2 and 5)
- **Broker 3:** Leads 2 partitions (Partitions 0 and 4)

3.B Why did Kafka spread the partition leaders across all three brokers instead of putting them on one? Answer:
Kafka spreads partition leaders evenly across all available brokers to **balance the client read/write load** across the cluster. If all leaders were on a single broker, that node would become a severe performance bottleneck and a single point of failure for active client traffic, while the other brokers would sit idle.



TASK 4 — STORAGE: Find Partitions and Segments on Disk

4 Locate Partition Folders and Segment Files on a Broker ⏱ 5 min

Kafka Ops SOP — Day 4 Lab 1

A partition replica is simply a folder on a broker's disk. Inside it, data is written to segment files with matching index files. You will compare what kafka-1 and kafka-2 store.

⌨ Task 4 — Look inside the brokers' log directories

```
# Step 1: which partition folders does each broker hold?
docker exec kafka-1 ls /tmp/kraft-combined-logs
docker exec kafka-2 ls /tmp/kraft-combined-logs

# Step 2: inside one partition folder
docker exec kafka-1 ls /tmp/kraft-combined-logs/orders-0

# Step 3: sizes per partition replica, reported by Kafka itself
./k kafka-log-dirs.sh --bootstrap-server $BS --describe --topic-list orders
```

⌨ Expected Output

```
Step 1:  each broker shows __cluster_metadata-0, orders-0 … orders-5, payments-0 … payments-2, meta.properties ...
         audit-log-0 appears on ONE broker only (replication factor 1)
Step 2:  00000000000000000000.log  00000000000000000000.index  00000000000000000000.timeindex
         leader-epoch-checkpoint  partition.metadata
Step 3:  JSON listing each broker's log dir with partition sizes (0 bytes — nothing produced yet)
```

4.A Which broker holds audit-log-0? How many orders-* folders does each broker hold, and why that number? Answer:

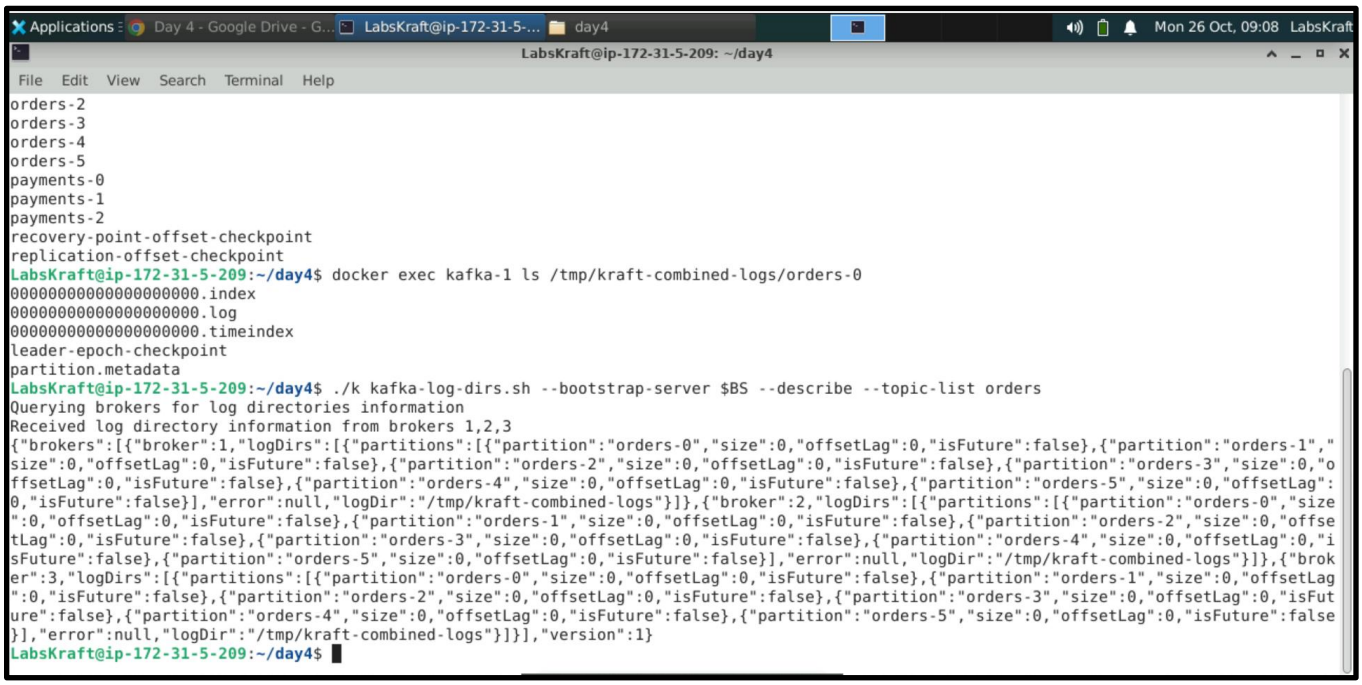
`audit-log-0` Location: It is held on **`kafka-2`** (as seen in the directory listing output where `audit-log-0` appears under kafka-2's log directories).

`orders-*` Folders Per Broker: Each broker holds **all 6** partition folders (`orders-0` through `orders-5`).

Reason: The `orders` topic was created with a replication factor of 3 in a 3-broker cluster, which means a full replica of every single partition (`0` to `5`) is stored across every available broker for high availability.

4.B What does the long number 00000000000000000000 in the segment file name represent? Answer: It represents the base offset (the starting message offset) of the first message stored in that log segment file, padded with leading zeros to 20 digits.





5. VALIDATION — CONFIRM FULL READINESS

Do not close the incident or call for instructor sign-off until ALL 8 validation steps below are completed and recorded.

Figure: Lab 1 — Kafka Architecture Validation Checklist

5.1 Validation Task Set

Validation Check 1 — Three nodes running

Validation 1

```
docker compose ps

# PASS criterion: kafka-1, kafka-2, kafka-3 all Up
```

V1: Are all three containers up? Answer:

```
LabsKraft@ip-172-31-5-209:~/day4$ docker compose ps
NAME      IMAGE                  COMMAND                  SERVICE   CREATED          STATUS          PORTS
kafka-1   apache/kafka:4.3.1     "/__cacert_entrypoin…"   kafka-1   39 minutes ago   Up 39 minutes   0.0.0.0:29092->9092/tcp, [::]:29092->9092
/tcp
kafka-2   apache/kafka:4.3.1     "/__cacert_entrypoin…"   kafka-2   39 minutes ago   Up 39 minutes   0.0.0.0:39092->9092/tcp, [::]:39092->9092
/tcp
kafka-3   apache/kafka:4.3.1     "/__cacert_entrypoin…"   kafka-3   39 minutes ago   Up 39 minutes   0.0.0.0:49092->9092/tcp, [::]:49092->9092
/tcp
LabsKraft@ip-172-31-5-209:~/day4$
```

Validation Check 2 — Quorum healthy

Validation 2

```
./k kafka-metadata-quorum.sh --bootstrap-server $BS describe --status

# PASS criterion: a LeaderId and three CurrentVoters
```

V2: Does the quorum show a leader and three voters? Answer:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-metadata-quorum.sh --bootstrap-server $BS describe --status
ClusterId:              4L6g3nShT-eMCtK--X86sw
LeaderId:               1
LeaderEpoch:            1
HighWatermark:          4868
MaxFollowerLag:         0
MaxFollowerLagTimeMs:   0
CurrentVoters:          [{"id": 1, "endpoints": ["CONTROLLER://kafka-1:9093"]}, {"id": 2, "endpoints": ["CONTROLLER://kafka-2:9093"]}, {"id"
: 3, "endpoints": ["CONTROLLER://kafka-3:9093"]}]
CurrentObservers:       []
LabsKraft@ip-172-31-5-209:~/day4$
```

Validation Check 3 — Topics created

Validation 3

```
./k kafka-topics.sh --bootstrap-server $BS --list

# PASS criterion: audit-log, orders, payments (too-many-copies absent)
```

V3: Do exactly the three intended topics exist? Answer:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-topics.sh --bootstrap-server $BS --list
audit-log
orders
payments
LabsKraft@ip-172-31-5-209:~/day4$
```

Validation Check 4 — orders layout correct

Validation 4

```
./k kafka-topics.sh --bootstrap-server $BS --describe --topic orders | head -1

# PASS criterion: PartitionCount: 6 and ReplicationFactor: 3
```

V4: Does orders have 6 partitions and RF 3? Answer:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-topics.sh --bootstrap-server $BS --describe --topic orders | head -1
Topic: orders   TopicId: 6jk11ykdTr-hyhwQT1EwdQ PartitionCount: 6       ReplicationFactor: 3    Configs: min.insync.replicas=1
LabsKraft@ip-172-31-5-209:~/day4$
```

Validation Check 5 — payments durability setting

Validation 5

```
./k kafka-topics.sh --bootstrap-server $BS --describe --topic payments | head -1

# PASS criterion: Configs: min.insync.replicas=2
```

V5: Is min.insync.replicas=2 set on payments? Answer:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-topics.sh --bootstrap-server $BS --describe --topic payments | head -1
Topic: payments TopicId: KnDBXl3XSumm1K5EwgdgmA PartitionCount: 3       ReplicationFactor: 3    Configs: min.insync.replicas=2
LabsKraft@ip-172-31-5-209:~/day4$
```

Validation Check 6 — All replicas in sync

Validation 6

```
./k kafka-topics.sh --bootstrap-server $BS --describe --under-replicated-partitions

# PASS criterion: no output - nothing is under-replicated
```

V6: Is the under-replicated list empty? Answer:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-topics.sh --bootstrap-server $BS --describe --under-replicated-partitions
LabsKraft@ip-172-31-5-209:~/day4$
```

Validation Check 7 — Partition replicas on disk

Validation 7

```
docker exec kafka-3 ls /tmp/kraft-combined-logs | grep -c '^orders-'

# PASS criterion: 6 - every broker holds a replica of every orders partition (RF 3 on
3 brokers)
```

V7: Does kafka-3 hold all 6 orders replicas? Answer:

```
LabsKraft@ip-172-31-5-209:~/day4$ docker exec kafka-3 ls /tmp/kraft-combined-logs | grep -c '^orders-'
6
LabsKraft@ip-172-31-5-209:~/day4$
```

General - RPS Data

Validation Check 8 — Full lab record complete

⌨ Validation 8 — Final completeness check

```
# Confirm every answer box (PRE-LAB, 1.A through 4.B) is completed
# Leave the cluster RUNNING - Labs 2-5 use it
# PASS criterion: every answer box holds real output (IDs, leaders, folder names)
```

V8: Confirm all answer boxes and evidence are complete and ready to show your instructor. Answer: _____

✅ STOP HERE — Show your completed validation record to your instructor before proceeding to Section 7.

General - RPS Data

6. TROUBLESHOOTING — COMMON ISSUES & FIXES

Issue / Error	Cause	Diagnosis	Fix
permission denied while trying to connect to the Docker daemon	Your user is not in the docker group	groups | grep docker	Ask the instructor to add you to the docker group (then log out and in)
Bind for 0.0.0.0:29092 failed: port is already allocated	Another Kafka or old container uses the port	docker ps -a	Stop/remove the old container, then docker compose up -d
Timed out waiting for a node assignment / connection refused	Commands run before the brokers finished starting	docker compose logs | grep 'Kafka Server started' shows fewer than 3 lines	Wait 20 seconds and retry
--bootstrap-server requires a value / Missing required argument	BS is not set in this terminal	echo $BS prints nothing	export BS=kafka-1:19092,kafka-2:19092,kafka-3:19092
Error: No such container: kafka-1	Cluster not started, or started from another folder	docker compose ps from ~/day4	cd ~/day4 && docker compose up -d

General - RPS Data

7. INSTRUCTOR DEBRIEF — 8 MINUTES

7.1 Discussion Questions

Discussion 1 — Kafka Components

Without looking at notes: draw producers, brokers, partitions with leader and followers, consumers, and the controller quorum.

Which component would you check first if no topic could be created? And if one partition could not be read?

Key answer to explore: topic creation needs the controller quorum; reading a partition needs that partition's leader broker.

Discussion 2 — KRaft vs ZooKeeper

Kafka 4.x removed ZooKeeper. What did that remove from an operator's checklist?

Why do the three nodes in this lab act as both brokers and controllers, and why might production separate them?

Key insight: one system to operate; isolated controllers protect metadata from broker load in large clusters.

Discussion 3 — Partitions and Replication

orders has 6 partitions and RF 3. How many partition replicas exist in total, and how many leaders?

What does audit-log risk with replication factor 1?

Key answer: 18 replicas, 6 leaders; RF 1 means one broker failure makes the topic unavailable and can lose data.

Discussion 4 — Production Considerations

Why should automatic topic creation be disabled in production, and topics created through reviewed scripts instead?

Answer: auto-created topics use default partitions and replication, which rarely match the workload or durability needs.

7.2 Common Mistakes — Learn From These

General - RPS Data

Mistake	Why It Happens	Consequence	Prevention
Running commands before brokers are ready	Containers look Up immediately	Timeouts mistaken for config errors	Wait for Kafka Server started on every node
Replication factor 1 for important data	Fewer brokers in development	Data loss or unavailability on one failure	RF 3 + min.insync.replicas 2
Too few partitions	Default of 1 partition	Only one consumer can read in parallel	Size partitions for consumer parallelism
Deleting files in log directories	Disk looks full	Corrupted partitions	Use retention settings and kafka tools
Forgetting the bootstrap variable in a new terminal	Environment variables are per shell	Confusing missing-argument errors	export BS=... in every terminal

General - RPS Data

8. EXTENSION TASKS (Advanced Learners)

Extension A — Topic Configuration

Run ./k kafka-configs.sh --bootstrap-server $BS --describe --entity-type topics --entity-name orders --all and find retention.ms and segment.bytes.

Change retention on audit-log: --alter --add-config retention.ms=3600000, then describe it again.

Key question: what happens to old segments when retention.ms expires?

Extension B — Connect From the Host

From your VM's shell (not ./k), explain why clients there must use localhost:29092 instead of kafka-1:19092.

Find the PLAINTEXT_HOST listener in docker-compose.yml and the advertised listener for each node.

Research: what goes wrong when advertised.listeners is set incorrectly?

Extension C — Isolated Controllers

Research Kafka's isolated mode (separate controller nodes and broker nodes).

Sketch how docker-compose.yml would change for 3 controllers and 3 brokers.

Explain when an organisation should choose isolated over combined mode.

General - RPS Data

9. LAB COMPLETION & SIGN-OFF

9.1 Submission Checklist

✓	Completion Criterion	Status
☐	Three-node KRaft cluster started; readiness confirmed in logs	☐ Pass / ☐ Fail
☐	Controller quorum inspected: cluster ID, leader and three voters recorded	☐ Pass / ☐ Fail
☐	orders (6×RF3), payments (3×RF3, min ISR 2) and audit-log (1×RF1) created and described	☐ Pass / ☐ Fail
☐	Partition folders and segment files located on brokers	☐ Pass / ☐ Fail
☐	All 8 validation checks completed and recorded	☐ Pass / ☐ Fail
☐	All observation/answer boxes in this guide completed	☐ Pass / ☐ Fail

9.2 Grading Criteria

Assessment Area	Criteria	Marks	Weight
Task 1 — Start Cluster	Cluster started; readiness evidenced	15	15%
Task 2 — Quorum	Quorum read correctly; fault tolerance explained	20	20%
Task 3 — Topics	Topics created as specified; leaders and ISR interpreted	30	30%
Task 4 — Storage	Partition folders and segments located and explained	25	25%
Validation & Documentation	All 8 validation checks completed; answers complete and accurate	10	10%
TOTAL		100	100%

9.3 Instructor Sign-Off

General - RPS Data

Field	Value
Lab Result	☐ PASS ☐ PARTIAL ☐ REPEAT REQUIRED | Score: _____ / 100
Instructor Name	_____
Instructor Signature & Date	_____
Remarks / Feedback	_____ _____
Completion Date & Time	_____

General - RPS Data

10. QUICK REFERENCE — KAFKA CLUSTER & TOPIC COMMAND CARD

Bookmark this page. Every command you need in SOP order.

Category	Command	Purpose
Setup	cp -r /labfiles/day4 ~/day4 && cd ~/day4	Your own working copy
Setup	export BS=kafka-1:19092,kafka-2:19092,kafka-3:19092	Bootstrap list (every new terminal)
Cluster	docker compose up -d / docker compose ps	Start and check the cluster
Cluster	docker compose logs | grep "Kafka Server started"	Readiness of each node
Quorum	./k kafka-metadata-quorum.sh --bootstrap-server $BS describe --status	Controller leader and voters
Topics	./k kafka-topics.sh --bootstrap-server $BS --create --topic t --partitions p --replication-factor r	Create a topic
Topics	./k kafka-topics.sh --bootstrap-server $BS --list	List topics
Topics	./k kafka-topics.sh --bootstrap-server $BS --describe --topic t	Leaders, replicas, ISR, configs
Health	./k kafka-topics.sh --bootstrap-server $BS --describe --under-replicated-partitions	Partitions missing replicas
Storage	docker exec kafka-1 ls /tmp/kraft-combined-logs	Partition folders on a broker
Storage	./k kafka-log-dirs.sh --bootstrap-server $BS --describe --topic-list t	Replica sizes per broker